CPT205
Computer Graphics

Name: Yifei Li

ID: 2364053

Degree programme: DMT

1. Program functions and related graphic technologies

1.1 Creation of Geometric Figures and Hierarchical Modeling

1.1.1 Comprehensive Application of Geometric Primitives

(1) Cube drawing:The main body of the building uses glutSolidCube()

```
GlutSolidCube (1.0 f); // Basic cube primitives
glScalef(width, height, depth);    // Size transformation
```

(2) Sphere drawing:Stars use glutSolidSphere()

```
glutSolidSphere(currentSize * 0.3f, 8, 8);
```

(3) Teapot drawing:Use glutSolidTeapot() as the scene decoration

```
GlutSolidTeapot (1.0); // Classic teapot graphic element
glScalef(0.6f, 0.6f, 0.6f); // Size adjustment
```

(4) Drawing of two-dimensional quadrilaterals (for mapping) :The texture plane uses GL_QUADS

```
glBegin(GL_QUADS);
glTexCoord2f(0.0f, 0.0f); glVertex3f(-width/2, -height/2, 0.0f);
```

(5) Line drawing:The tracks and lines use GL_LINES

```
glBegin(GL_LINES);                // Start drawing straight lines
glVertex3f(LINE1_START_X, LINE1_START_Y, LINE1_START_Z);    // Starting point
glVertex3f(LINE1_END_X, LINE1_END_Y, LINE1_END_Z);          // Endpoint
```

1.1.2 Hierarchical Modeling technology

Robot hierarchical structure：

Divide the robot into five levels: the trunk, the left arm, the right arm, the left leg, and the right leg. Under each level, connect the forearm, the upper arm, the hand, the thigh, the calf, the foot, etc. to establish its own structure.

```
// Level 2: Left arm (big arm -> small arm -> hand)
glPushMatrix();
glTranslatef(-2.5f, 5.0f, 0.0f); // Left shoulder position
Big arms
glPushMatrix();
glScalef(1.0f, 2.5f, 1.0f);
GlutSolidCube (1.0 f);
```

1.2 Application of transformation technology

(1) Translation transformation: glTranslatef() is used to locate all objects

```
glTranslatef(BLD1_X, BLD1_Y, BLD1_Z);    // Building positioning
glTranslatef(-3.0f, 8.5f, 7.4f); // Robot positioning
```

(2) Rotation transformation: glRotatef() is used for building rotation and aircraft orientation

```
// Building rotation (25 degrees)
glRotatef(25.0f, 0.0f, 1.0f, 0.0f); // Rotate around the Y-axis
```

(3) Scaling transformation: glScalef() is used to adjust the size of objects

```
// Uniform scaling
glScale(0.6f, 0.6f, 0.6f); // Overall scaling of the robot
```

1.3 View and projection system

1.3.1 Camera System Design

Fixed-position camera：View rotation control, achieve 360-degree all-round view observation by dragging with the left mouse button. The system adopts a first-person fixed camera, allowing users to freely explore the entire urban landscape.

```
// Camera parameters
float cameraX = 1.5f, cameraY = 12.5f, cameraZ = 15.5f; // Fixed position
float camera aw = 15.0f, camera pitch = -10.0f; // Variable orientation
```

1.3.2 Observation Matrix Calculation

```
/ Set the observation matrix
gluLookAt(cameraX, cameraY, cameraZ, // camera position
centerX, centerY, centerZ, // observation points
0.0f, 1.0f, 0.0f); // Upward direction
```

1.3.3 Projection System

```
// Perspective projection Settings
void reshape(int width, int height) {
glViewport(0, 0, width, height);          // Viewport Settings
glMatrixMode(GL_PROJECTION);              // Switch to the projection matrix
```

1.4. Lighting and Material Systems

1.4.1 Color and Material Definitions

Grayscale architecture: Use different grayscale values to distinguish the colors of buildings

High-speed rail: Dark blue (0.0, 0.0, 0.1)

Cable car: Red wine (0.25, 0.0, 0.05)

Aircraft: Silver (0.75, 0.75, 0.75)

Robot color: A combination of dark gray, light gray and black

1.4.2 Starlight illumination

```
// Star multi-layer luminescence effect (simulating self-luminous material)
glDisable(GL_DEPTH_TEST);    // Disable depth testing (stars are always visible)
glEnable(GL_BLEND);          // Enable blending
glBlendFunc(GL_SRC_ALPHA, GL_ONE);    // Accumulate the luminescence effect
```

1.5 Texture Mapping technology

1.5.1 Multi-texture Loading System:

11 different textures are applied to various surfaces

Set the texture ID, read the width and height information of the BMP, calculate the offset and length to load the BMP texture, and then read the pixel data in BGR format.

```
// Read and load the subway wall texture
subwayWallTexture = LoadTexture("subway_wall.bmp", subwayWallPixelData);
```

1.5.2 Texture Parameter Settings

Generate a specific texture ID, bind the corresponding texture, then filter the texture parameters, set the texture surround mode, and upload the texture data (BGR->RGB conversion)

```
GLuint subwayWallTexture;        // Subway wall texture
```

```
GLubyte* subwayPixelData = nullptr;        // Metro ground texture pixel data pointer
```

1.5.3 Texture Coordinate Mapping

Texture mapping is carried out through point-to-point

```
// Lower left corner: Texture coordinates (0,0) -> vertex coordinates
glTexCoord2f(0.0f, 0.0f);
glVertex3f(-BLD1_WIDTH/2, -BLD1_HEIGHT/2, BLD1_DEPTH/2);
```

1.6 Animation System Design

1.6.1 Unify the timer framework

All animation elements are uniformly time-managed and run smoothly at a frame rate of 60FPS without user intervention

```
void timer(int value) {
updatePlane();                    // Update the position of the aircraft
```

1.6.2 Path Animation Technology

Bezier curve path: It is used to simulate the flight of real aircraft and adjust the flight attitude in real time

```
const Point3D pathPoints[] = {
{45.0f, 35.0f, -90.0f}, // starting point
{1.0f, 5.0f, -23.0f}, // control points
{4.0f, 25.0f, 20.0f} // Endpoint
};
```

(2) Linear interpolation animation: It is used to simulate the movement of cable cars. Three cable cars run asynchronously along a straight track, forming a continuous flow of vehicles and a high-speed rail system, and automatically loop along a rectangular track, with real-time calculation of position and orientation

```
void calculateCableCarPosition(int index, float* x, float* y, float* z) {
float progress = cableCarProgress[index];
*x = LINE1_START_X + (LINE1_END_X - LINE1_START_X) * progress;
```

1.6.3 Starry Sky Dynamic Effect

Pulsating Starry Sky System: 500 stars are randomly distributed, featuring diversity in color, brightness and size, achieving a natural twinkling effect.

```
// Star Pulse Algorithm (Sine Wave)
void updateStars() {
static float time = 0.0f;
time += 0.016f; // Time accumulation
```

1.7 Interactive System Design

1.7.1 Keyboard Interaction

The interaction between the robot and the teapot can be achieved by pressing the 'A' key

```
f (key == 'a') {// The A key controls the robot
leftArmRaised = ! leftArmRaised;    // Switch the left arm position
teapotRaised = ! teapotRaised;       // Synchronize the status of the teapot
glutPostRedisplay();                 // Update display
```

1.7.2 Mouse Interaction

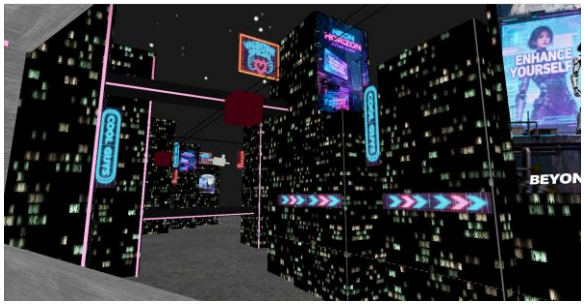
Move the perspective through the mouse

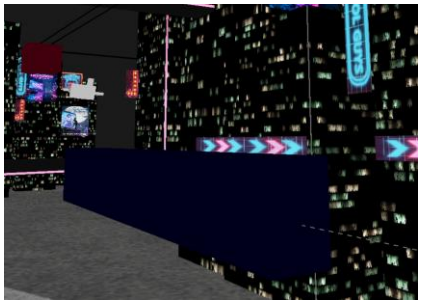
```
void mouseMotion(int x, int y) {
if (mouseLeftDown) {
float deltaX = x - lastMouseX;
float deltaY = y - lastMouseY;
cameraYaw += deltaX * 0.1f; // Horizontal rotation
cameraPitch -= deltaY * 0.1f; // Vertical rotation (note the symbol)
```

2. Operating Instructions

(1) The initial screen and other visible scenes can be controlled to rotate the perspective by dragging with the left mouse button
Horizontal dragging: Rotate the Angle of view (yaw Angle) left and right
Vertical drag: Rotate the viewing Angle (pitch Angle) up and down

Viewing Angle limit: The pitch Angle is limited within ±85 degrees



(2) "A/a" key:
Control the robot to pick up and put down the teapot
Realize the interaction between the robot and the teapot
Key 'ESC' : Safely exit the program and automatically release all texture memory

(3) High-speed rail, cable car, aircraft, stars, building poster stickers